



Complete Guide

Mardas MD2PDF Guide

Complete usage manual and feature sample

Install, configure, and render professional Markdown PDFs.

The generated PDF demonstrates cover pages, TOC, math, code, tables, images, HTML, page breaks, and footnotes.

AUTHORS

Mardas MD2PDF Team
(Documentation)
Meraj Rastegar
(mragetsars@yahoo.com)

DATE

2026-05-20

INSTITUTION

Mardas Lab

COURSE

Markdown Publishing

VERSION

1.3.1

STATUS

Stable

KEYWORDS

Markdown
PDF
Persian
English
RTL/LTR
MathJax
Playwright

Table of Contents

- 1 Getting Started
 - 1-1 Installation
 - 1-2 First PDF
 - 1-3 When to use the GUI
- 2 Front Matter and Cover Pages
 - 2-1 Common fields
 - 2-2 Cover behavior
- 3 Language, Direction, and Typography
 - 3-1 Mixed text example
 - 3-2 Direction control tips
- 4 Table of Contents
- 5 Markdown Features
 - 5-1 Tables
 - 5-2 Task lists
 - 5-3 Blockquotes
 - 5-4 Callouts
- 6 MathJax
- 7 Code Highlighting
- 8 Images and Safe HTML
- 9 Page Flow, Watermarks, and Themes
 - 9-1 Manual page breaks
 - 9-2 Watermarks
 - 9-3 Themes
- 10 CLI Reference
- 11 Troubleshooting
 - 11-1 Missing browser
 - 11-2 Missing images
 - 11-3 Math appears as raw TeX
 - 11-4 Need to debug layout

[12](#) Footnotes

[13](#) Final Checklist

MARDAS GUIDE

Getting Started

Mardas MD2PDF converts Markdown files into polished PDFs by using a browser-grade rendering pipeline:

TEXT

```
Markdown -> typographic HTML -> Chromium PDF
```

This approach keeps Markdown authoring simple while giving the renderer strong control over printed layout, MathJax output, local images, cover pages, page breaks, and mixed RTL/LTR typography.

Note

This file is both a user guide and a visual test case. Render it to PDF to review most of the features supported by the project.

■ Installation

Clone the repository, create a virtual environment, install the package, and install Chromium for Playwright:

BASH

```
git clone https://github.com/mragetsars/Mardas-MD2PDF.git
cd Mardas-MD2PDF
python -m venv .venv
source .venv/bin/activate
pip install -e .
python -m playwright install chromium
```

Windows PowerShell:

POWERSHELL

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e .
python -m playwright install chromium
```

For development and tests:

BASH

```
pip install -e .[dev]
pytest
```

■ First PDF

Create a PDF with the default modern theme:

BASH

```
mrs-md2pdf input.md -o output.pdf
```

Create a report with a table of contents:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --toc-depth 4 --theme modern
```

Create a book-like PDF where the TOC ends on its own page and each top-level heading starts on a new page:

BASH

```
mrs-md2pdf input.md -o output.pdf \
  --toc \
  --toc-depth 4 \
  --toc-page-break \
  --h1-page-break \
  --theme textbook-light
```

■ When to use the GUI

Launch the local editor/exporter:

BASH

```
mrs-md2pdf-gui
```

The GUI is useful when you want to edit Markdown, preview the document, choose a theme, set export options, attach local image files/folders, and export the PDF without remembering CLI flags.

Front Matter and Cover Pages

Front matter is optional YAML placed at the beginning of a Markdown file. It controls the cover page, PDF metadata, language, direction, and many document details.

YAML

```
---
title: "Mardas MD2PDF Complete Guide"
subtitle: "Feature tour and usage manual"
authors:
  - name: "Mardas MD2PDF Team"
    role: "Documentation"
  - name: "Meraj Rastegar"
    email: "mrage@sars@yahoo.com"
summary: |
  Multiline summaries are preserved on the cover.
  Blank lines create separate paragraphs.
institution: "Mardas Lab"
version: "1.3.1"
keywords: [Markdown, PDF, RTL/LTR, MathJax]
cover_label: "Complete Guide"
lang: en
dir: ltr
---
```

Common fields

Field	Purpose
<code>title</code>	Cover title and PDF metadata title.
<code>subtitle</code>	Optional text under the title.
<code>author</code> / <code>authors</code>	Single or multiple authors. Author objects may include <code>name</code> , <code>email</code> , <code>affiliation</code> , and <code>role</code> .
<code>summary</code> / <code>description</code>	Cover summary and PDF metadata subject. Multiline YAML blocks are supported.
<code>date</code> , <code>version</code> , <code>status</code>	Optional cover cards.
<code>institution</code> , <code>course</code> , <code>department</code> , <code>supervisor</code> , <code>group</code> , <code>student_id</code>	Optional academic/report metadata cards.
<code>keywords</code> / <code>tags</code>	Cover keyword card and PDF metadata keywords.
<code>cover_label</code>	Small label above the cover title.
<code>cover_logo</code> / <code>logo</code>	Custom cover logo path relative to the Markdown file.
<code>lang</code>	Built-in UI language, such as <code>en</code> or <code>fa</code> .
<code>dir</code>	Document shell direction: <code>auto</code> , <code>ltr</code> , or <code>rtl</code> .

Cover behavior

The cover is rendered separately from the main document. That means:

- the cover is not counted in content page numbering;
- header/footer numbering starts after the cover;
- watermarks are applied to content pages only;
- the cover can use full-bleed theme backgrounds.

Disable the cover:

BASH

```
mrs-md2pdf input.md -o output.pdf --no-cover
```

Use a custom logo:

BASH

```
mrs-md2pdf input.md -o output.pdf --cover-logo ./assets/logo.png
```

Hide the logo while keeping the cover:

BASH

```
mrs-md2pdf input.md -o output.pdf --no-cover-logo
```

Language, Direction, and Typography

`lang: en` creates an English/LTR document shell, English cover labels, English callout titles, and a `Table of Contents` heading. `lang: fa` creates a Persian/RTL document shell, Persian labels, and `فهرست مطالب`.

The direction resolution order is:

1. CLI `--dir rtl|ltr|auto`
2. front matter `dir`, `direction`, `text_direction`, or `document_direction`
3. language-derived default from `lang`
4. automatic detection from the Markdown body

■ Mixed text example

English technical writing can include Persian terms such as `راست به چپ`, `چپ به راست`, and `فونت فارسی` without breaking the surrounding sentence. Persian documents can also include English identifiers such as `Playwright`, `MathJax`, `PDF`, and `RTL/LTR` inside one paragraph.

Inline code remains readable: `mrs-md2pdf input.md -o output.pdf --toc`.

■ Direction control tips

- Use `lang: en` for English guides, API documents, and reports.
- Use `lang: fa` for Persian reports and course notes.
- Use `dir: auto` when you want the converter to infer the shell direction.
- Use `--dir ltr` or `--dir rtl` when a CI/CD job must force a specific layout.

Table of Contents

Enable the TOC with:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc
```

Control depth:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --toc-depth 3
```

Start the body after the TOC on a new page:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --toc-page-break
```

The TOC is built from Markdown headings and keeps inline math readable when headings contain formulas such as $E = mc^2$ or ϵ .

Markdown Features

■ Tables

Feature	Status	Notes
Mixed RTL/LTR text	✓	Paragraphs, headings, list items, and table cells receive direction-aware handling.
Local images	✓	Markdown and safe HTML images are embedded as data URIs.
MathJax	✓	Inline and display formulas use separate scaling.
Code highlighting	✓	Fenced and indented code blocks are highlighted with Pygments.
Footnotes	✓	Multiline footnotes with Markdown content are supported.
Raw HTML sanitizer	✓	Unsafe tags and event handlers are removed by default.

■ Task lists

- ✓ Write Markdown.
- ✓ Configure front matter.
- ✓ Render a PDF.
- ☐ Review the output in more than one viewer if the document is critical.

■ Blockquotes

Good PDF publishing is not just text conversion. Typography, page flow, figure stability, contrast, and predictable rendering all matter.

■ Callouts

Tip

Use `--debug-html output.html` when you need to inspect the exact HTML sent to Chromium.

Warning

Use `--unsafe-html` only for trusted local Markdown because it disables the built-in sanitizer.

MathJax

Inline math should match the surrounding line height: $E = mc^2$, $\Sigma = I \cdot \epsilon$, and $T = 500$ should sit naturally inside a paragraph.

Display math gets more space and a larger visual scale:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

A matrix example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \det(A) = -2$$

An aligned equation block:

$$\begin{aligned} \text{precision} &= \frac{TP}{TP + FP} \\ \text{recall} &= \frac{TP}{TP + FN} \end{aligned}$$

Code Highlighting

Fenced code blocks show the language label and syntax highlighting.

PYTHON

```
from dataclasses import dataclass

@dataclass
class Document:
    title: str
    lang: str = "en"

def render_message(doc: Document) -> str:
    return f"Rendering {doc.title} as a polished PDF"

print(render_message(Document("Mardas Guide")))
```

JAVASCRIPT

```
const items = ["Markdown", "Persian", "English", "MathJax", "PDF"];
const message = items.map((item, index) => `${index + 1}. ${item}`).join("\n");
console.log(message);
```

C

```
int setSeed(void);
int getRandomNumber(int n, int *buf);
int process_information(int pid);
int sort_numbers(const char *src_file);
```

Indented code blocks are also supported:

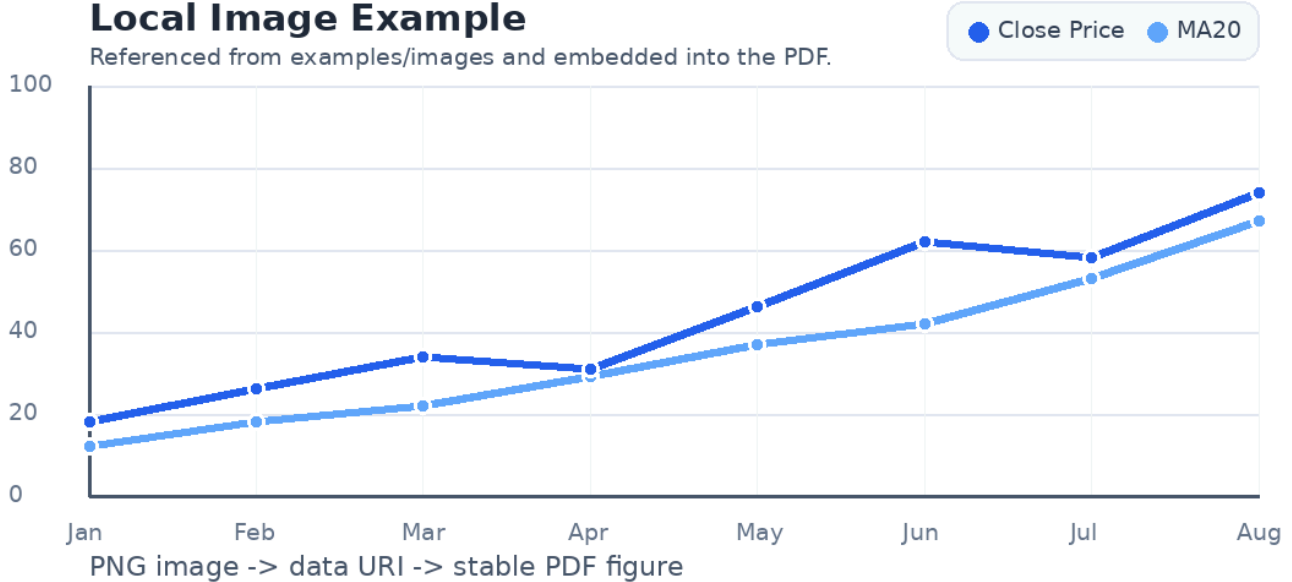
```
SELECT title, lang, version
FROM documents
WHERE renderer = 'mardas-md2pdf';
```

Images and Safe HTML

Markdown images are resolved relative to the Markdown file and embedded into the generated HTML/PDF.

Local Image Example

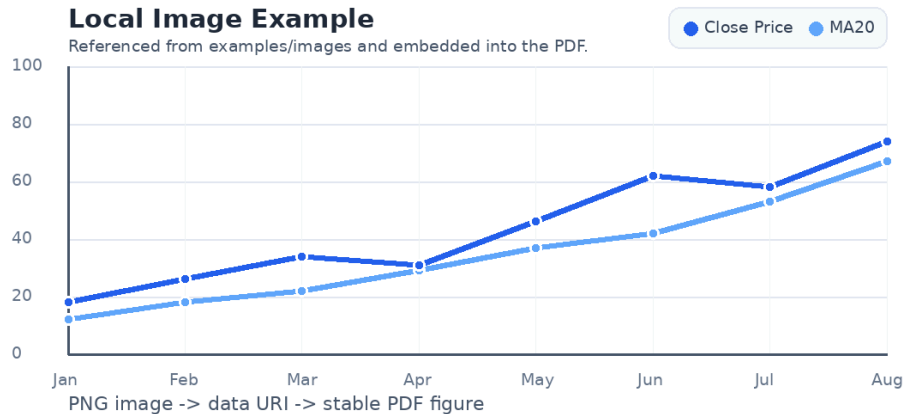
Referenced from examples/images and embedded into the PDF.



Safe raw HTML images can also be used when you need explicit sizing:

Local Image Example

Referenced from examples/images and embedded into the PDF.



Raw HTML is sanitized by default. The sanitizer keeps document-oriented elements such as `<div>`, `<span>`, `<table>`, `<figure>`, and `<img>`, while removing active content such as scripts, event handlers, iframes, forms, remote stylesheets, and unsafe URL schemes.

Page Flow, Watermarks, and Themes

Manual page breaks

A manual page break can be inserted with safe HTML:

HTML

```
<div class="md2pdf-page-break"></div>
```

Watermarks

Text watermark:

BASH

```
mrs-md2pdf input.md -o output.pdf --watermark "DRAFT"
```

Image watermark:

BASH

```
mrs-md2pdf input.md -o output.pdf \  
--watermark-image ./Mardas.png \  
--watermark-opacity 0.05 \  
--watermark-width 95mm
```

Watermarks are applied to content pages only, not to the cover.

Themes

Theme	Best for
modern	General documentation, proposals, software reports.

Theme	Best for
<code>textbook-light</code>	Course notes, long Persian/English educational PDFs.
<code>textbook-dark</code>	Screen reading and low-light review.
<code>academic</code>	Formal reports, university-style documents, thesis-like drafts.

CLI Reference

Option	Description
<code>input</code>	Input Markdown file.
<code>-o</code> , <code>--output</code>	Output PDF path.
<code>--title</code> , <code>--author</code> , <code>--description</code>	Override metadata from front matter.
<code>--toc</code> , <code>--toc-depth</code>	Enable/configure the Table of Contents.
<code>--toc-page-break</code> , <code>--h1-page-break</code>	Control print page flow.
<code>--theme</code>	Choose <code>modern</code> , <code>textbook-light</code> , <code>textbook-dark</code> , or <code>academic</code> .
<code>--page-size</code>	<code>A4</code> , <code>Letter</code> , <code>Legal</code> , or a CSS page size.
<code>--dir</code>	Force <code>auto</code> , <code>ltr</code> , or <code>rtl</code> .
<code>--margin-top</code> , <code>--margin-bottom</code> , <code>--margin-x</code>	Control page margins.
<code>--font-dir</code>	Directory containing local Vazirmatn font files.
<code>--chromium-path</code>	Custom Chromium/Chrome executable path.
<code>--debug-html</code>	Save the intermediate HTML.
<code>--no-cover</code> , <code>--cover-logo</code> , <code>--no-cover-logo</code>	Configure the cover.

Option	Description
<code>--watermark</code> , <code>--watermark-image</code>	Add watermarks.
<code>--no-header-footer</code>	Disable the footer.
<code>--no-mathjax</code>	Do not load MathJax.
<code>--unsafe-html</code>	Disable raw HTML sanitization.
<code>--timeout-ms</code>	Browser timeout in milliseconds.

Troubleshooting

Missing browser

Run:

BASH

```
python -m playwright install chromium
```

or pass a custom browser path:

BASH

```
mrs-md2pdf input.md -o output.pdf --chromium-path /path/to/chrome
```

Missing images

Check that image paths are relative to the Markdown file. For GUI exports, attach the image folder/files so the backend can embed them before rendering.

Math appears as raw TeX

Make sure MathJax is enabled. Avoid `--no-mathjax` unless you intentionally want raw math markers.

■ Need to debug layout

Generate intermediate HTML:

BASH

```
mrs-md2pdf input.md -o output.pdf --debug-html output.html
```

Footnotes

Footnotes are useful for technical notes and references. [pipeline](#)

Final Checklist

Before publishing an important PDF:

- ☒ Check the cover metadata.
- ☒ Check the TOC language and direction.
- ☒ Check a page containing math.
- ☒ Check a page containing code.
- ☒ Check local images.
- ☒ Check footer numbering after the cover.
- ☐ Review the final PDF visually.

1. Mardas MD2PDF intentionally uses Chromium for layout instead of drawing every paragraph directly on a PDF canvas. This [↔](#) gives the project strong support for CSS print rules, mixed direction text, MathJax SVG output, tables, local images, and syntax-highlighted code.

- Multiline footnotes are supported.
- Markdown inside footnotes is preserved.
- Inline code like `@page` remains readable.